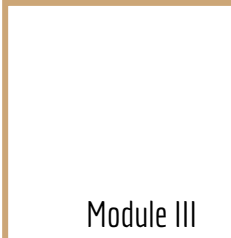


MODULE III





Module III

Arduino-I/O functions, Arduino PWM, Arduino Communication, LED Interfacing-Sinking & Sourcing methods of LED Connection, LED blinking, fading, Analog read, LED bar graph display, Seven Segment LED Display, interfacing sensors with Arduino-humidity, temperature, water detector, PIR, ultrasonic sensor, LDR, Interfacing Push Button Switch-Pull Up & Pull-Down Connection.



Arduino - I/O Functions

The pins on the Arduino board can be configured as either inputs or outputs.

We will explain the functioning of the pins in those modes.

It is important to note that a majority of Arduino analog pins, may be configured, and used, in exactly the same manner as digital pins.

Pins Configured as INPUT

- Arduino pins are set as **inputs by default**, so you don't need to use `pinMode()` unless you want to change them.
- Input pins are in a **high-impedance** state, meaning they don't draw much current from the circuit.
- They behave like they have a **very large resistor (about 100 megaohms)** in front of them.
- Because of this, it takes **very little current** to change their state from HIGH to LOW or vice versa.
- This is helpful when using **touch sensors** or reading light from an **LED as a sensor**.
- But if nothing is connected to an input pin, it can **pick up random signals** (like static or noise) and show **unpredictable results**.
- This can happen if a wire is connected to the pin but not to anything else.

Pins Configured as INPUT

```
pinMode(pin, OUTPUT)
```

Low-impedance — can **source/sink up to 40 mA**

Drives LEDs, simple circuits (with series resistor)

NOT suitable for motors, relays directly

Protect with 470Ω–1kΩ resistor if unsure

[pinMode() Function]

- Syntax: `pinMode(pin, mode);`
- `mode = INPUT, OUTPUT, or INPUT_PULLUP`

Example

```
int button = 5 ; // button connected to pin 5
int LED = 6; // LED connected to pin 6
int count=0;
int c=0;
Int
void setup() {
    pinMode(button , INPUT_PULLUP); // set the digital pin as input with pull-up resistor
    pinMode(button , OUTPUT); // set the digital pin as output
}

void loop () {
for(){

    If (digitalRead(button ) == HIGH) // if button pressed {
        count++;
    }
    delay(5);
    }
    serial.print(count);
}
```

Example

```
int button = 5 ; // button connected to pin 5  
int LED = 6; // LED connected to pin 6  
  
void setup() {  
  pinMode(button , INPUT_PULLUP); // set the digital pin as input with pull-up resistor  
  pinMode(button , OUTPUT); // set the digital pin as output  
}  
  
void loop () {  
  if (digitalRead(button ) == LOW) // if button pressed {  
    digitalWrite(LED,HIGH); // turn on led  
    delay(500); // delay for 500 ms  
    digitalWrite(LED,LOW); // turn off led  
    delay(500); // delay for 500 ms  
  }  
}
```

[digitalWrite() Function]

- Syntax: `digitalWrite(pin, value);`
- For OUTPUT pins: sets voltage to HIGH (5V) or LOW (0V)
- For INPUT pins: enables/disables pull-up resistor

Example :

```
int LED = 6; // LED connected to pin 6
```

```
void setup () {  
    pinMode(LED, OUTPUT); // set the digital pin as output  
}
```

```
void setup () {  
    digitalWrite(LED,HIGH); // turn on led  
    delay(500); // delay for 500 ms  
    digitalWrite(LED,LOW); // turn off led  
    delay(500); // delay for 500 ms  
}
```

[analogRead() Function]

- Syntax: `analogRead(pin);`
- Reads analog voltage (0–5V) on analog pins (A0–A5)
- Returns value from 0 (0V) to 1023 (5V)

Sensor Name	What It Measures	Typical Use
LM35 / TMP36	Temperature (°C)	Weather, environment, HVAC
LDR (Light Dependent Resistor)	Light intensity	Automatic lighting, light meters
Potentiometer	Rotation or variable resistance	Volume control, input knob
MQ Series (e.g., MQ-2, MQ-3)	Gas concentration (LPG, CO, alcohol)	Air quality, gas leak detection
Soil Moisture Sensor	Soil water level	Plant monitoring

Working of analogRead:

- The `analogRead()` function on Arduino Uno uses the microcontroller's built-in **10-bit ADC (Analog-to-Digital Converter)** to convert an analog voltage (0–5V) into a digital value (0–1023).
- Internally, it selects the appropriate analog input channel by configuring the **ADMUX** register.
- It then starts the conversion by setting a bit in the **ADCSRA** register and waits for the conversion to complete.
- The result is stored in two registers, **ADCL** and **ADCH**, which are combined to get the 10-bit value.
- Finally, `analogRead()` returns this value, representing the ratio of the input voltage to the reference voltage.

```
int analogPin = A3; // potentiometer wiper (middle terminal)  
  // connected to analog pin 3  
int val = 0; // variable to store the value read  
  
void setup() {  
  Serial.begin(9600); // setup serial  
}  
  
void loop() {  
  val = analogRead(analogPin); // read the input pin  
  Serial.println(val); // debug value  
}
```

Arduino - Pulse Width Modulation

Pulse Width Modulation or PWM is a common technique used to vary the width of the pulses in a pulse-train.

PWM has many applications such as controlling servos and speed controllers, limiting the effective power of motors and LEDs.

Basic Principle of PWM

Pulse width modulation is basically, a square wave with a varying high and low time. A basic PWM signal is shown in the following figure.

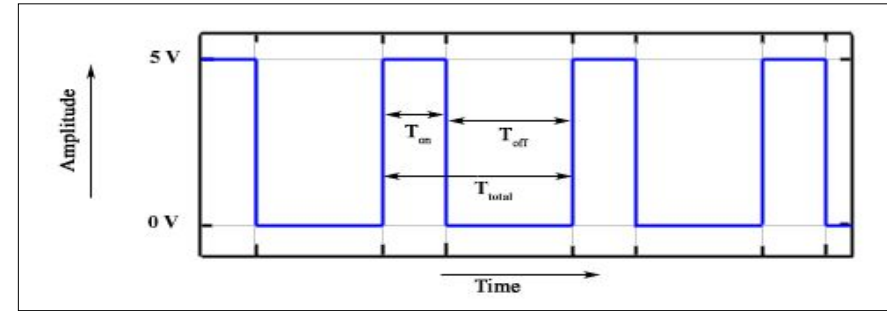
There are various terms associated with PWM –

On-Time – Duration of time signal is high.

Off-Time – Duration of time signal is low.

Period – It is represented as the sum of on-time and off-time of PWM signal.

Duty Cycle – It is represented as the percentage of time signal that remains on during the period of the PWM signal.



analogWrite() Function

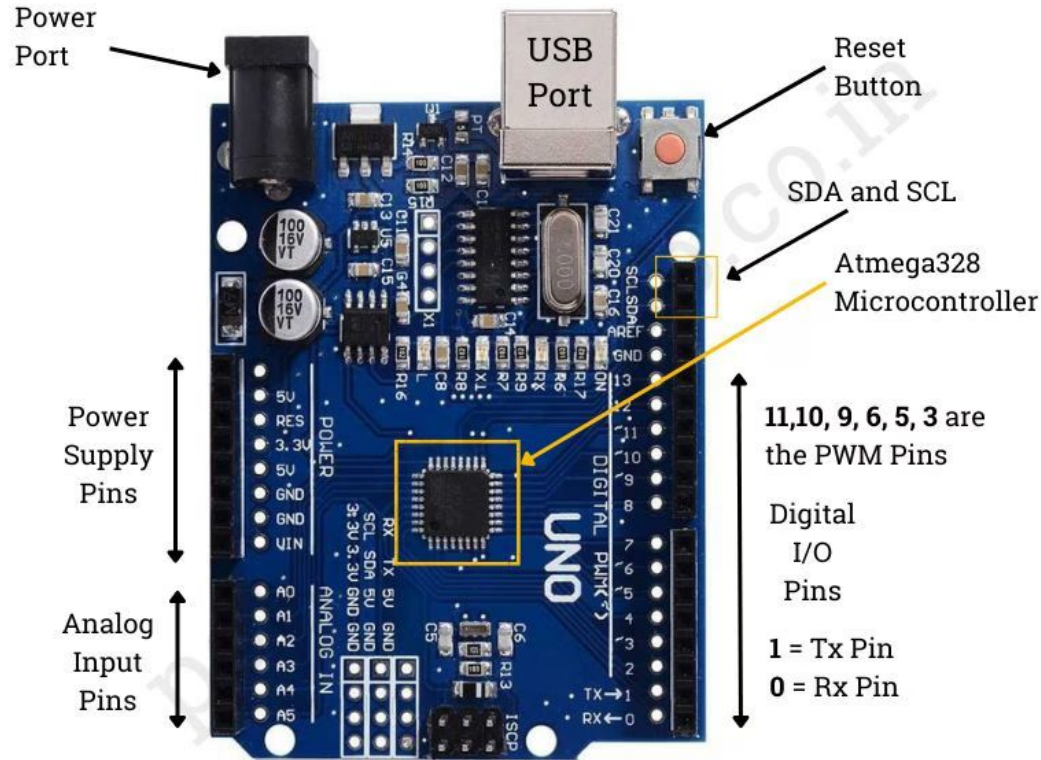
The **analogWrite()** function writes an analog value (PWM wave) to a pin.

It can be used to light a LED at varying brightness or drive a motor at various speeds.

After a call of the analogWrite() function, the pin will generate a steady square wave of the specified duty cycle until the next call to analogWrite() or a call to digitalWrite() or digitalWrite() on the same pin. The frequency of the PWM signal on most pins is approximately 490 Hz.

On the Uno and similar boards, pins 5 and 6 have a frequency of approximately 980 Hz. Pins 3 and 11 on the Leonardo also run at 980 Hz.





Arduino UNO Pinout


```
int ledPin = 9; // LED connected to digital pin 9
```

```
int analogPin = A3; // potentiometer connected to analog pin 3
```

```
int val = 0; // variable to store the read value
```

```
void setup() {
```

```
  pinMode(ledPin, OUTPUT); // sets the pin as output
```

```
}
```

```
void loop() {
```

```
  val = analogRead(analogPin); // read the input pin
```

```
  analogWrite(ledPin, (val / 4)); // analogRead values go from 0 to 1023,
```

```
  // analogWrite values from 0 to 255
```

```
}
```

Arduino - Communication

Hundreds of communication protocols have been defined to achieve this data exchange.

Each protocol can be categorized into one of the two categories:

parallel or serial.

Parallel Communication

Parallel connection between the Arduino and peripherals via input/output ports is the ideal solution for shorter distances up to several meters.

However, in other cases when it is necessary to establish communication between two devices for longer distances it is not possible to use parallel connection.

Parallel interfaces transfer multiple bits at the same time.

Parallel communication certainly has its advantages. It is faster than serial, straightforward, and relatively easy to implement. However, it requires many input/output (I/O) ports and lines. If you have ever had to move a project from a basic Arduino Uno to a Mega, you know that the I/O lines on a microprocessor can be precious and few. Therefore, we prefer serial communication, sacrificing potential speed for pin real estate.

Serial Communication Modules

Today, most Arduino boards are built with several different systems for serial communication as standard equipment.

Types of Serial Communications

Serial communication can be further classified as –

Synchronous – Devices that are synchronized use the same clock and their timing is in synchronization with each other.

Asynchronous – Devices that are asynchronous have their own clocks and are triggered by the output of the previous state.

It is easy to find out if a device is synchronous or not. If the same clock is given to all the connected devices, then they are synchronous. If there is no clock line, it is asynchronous.

For example, UART (Universal Asynchronous Receiver Transmitter) module is asynchronous.

Example :

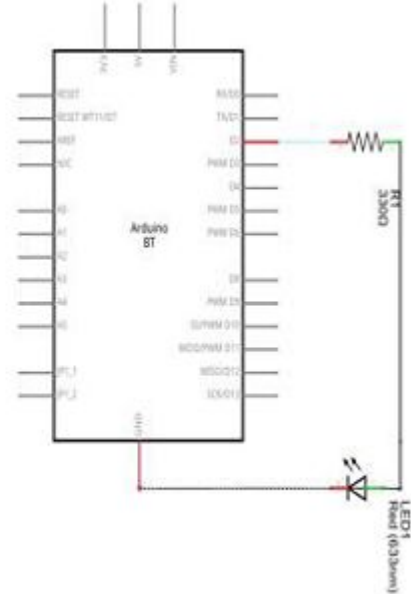
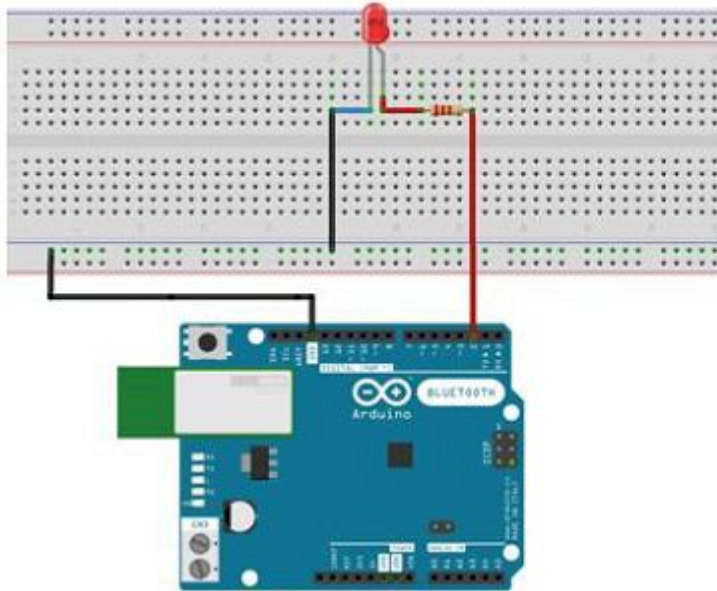
```
void setup() {  
  Serial.begin(9600); //set up serial library baud rate to 9600  
  Serial.println("hello world"); //print hello world  
}  
  
void loop() {  
  
}
```

Example :

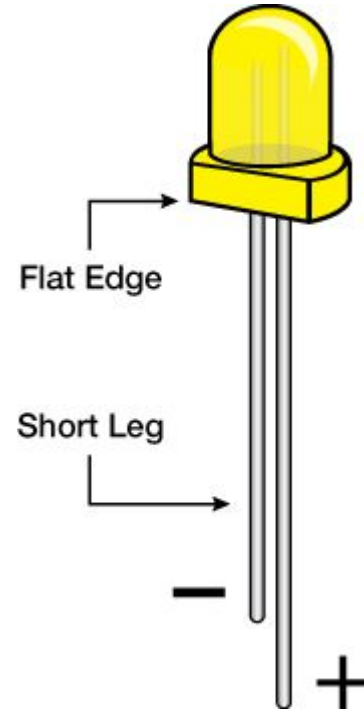
```
void setup() {  
    Serial.begin(9600); // Set up serial communication at 9600 baud  
}  
  
void loop() {  
    if (Serial.available()) { // If there is data available to read  
        char received = Serial.read(); // Read the incoming byte  
        Serial.print("I received: "); // Print a message  
        Serial.println(received); // Print the received character followed  
by a newline  
    }  
}
```

Arduino - Blinking LED

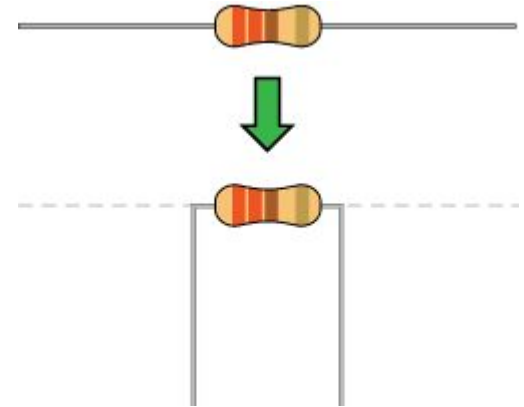
LEDs are small, powerful lights that are used in many different applications.



To find out the polarity of an LED, look at it closely. The shorter of the two legs, towards the flat edge of the bulb indicates the negative terminal.

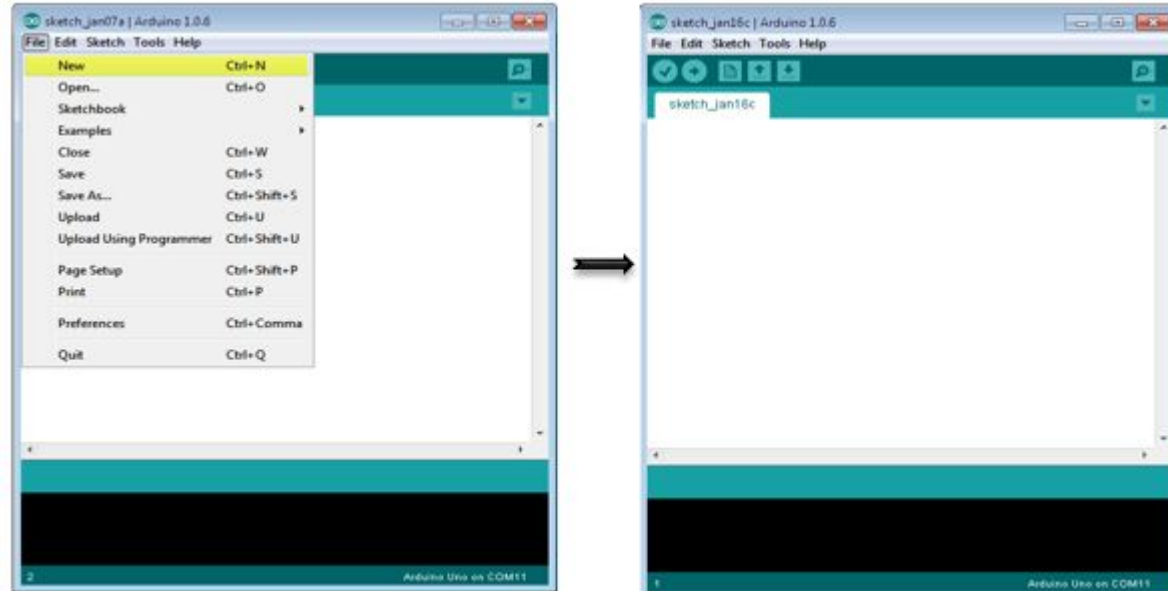


Components like resistors need to have their terminals bent into 90 angles in order to fit the breadboard sockets properly. You can also cut the terminals shorter.



Sketch

Open the Arduino IDE software on your computer. Coding in the Arduino language will control your circuit. Open the new sketch File by clicking New.



Arduino Code - Example for Digital write

```
/*  
  Blink  
  Turns on an LED on for one second, then off for one second, repeatedly.  
  */  
  
// the setup function runs once when you press reset or power the board  
  
void setup() { // initialize digital pin 2 as an output.  
  pinMode(2, OUTPUT);  
}  
  
// the loop function runs over and over again forever  
  
void loop() {  
  digitalWrite(2, HIGH); // turn the LED on (HIGH is the voltage level)  
  delay(1000); // wait for a second  
  digitalWrite(2, LOW); // turn the LED off by making the voltage LOW  
  delay(1000); // wait for a second  
}
```

Arduino - Fading LED

/ The analogWrite() function uses PWM, so if you want to change the pin you're using, be sure to use another PWM capable pin. On most Arduino, the PWM pins are identified with a "~" sign, like ~3, ~5, ~6, ~9, ~10 and ~11.*/*

*int led = 9; // the PWM pin the LED is attached to
int brightness = 0; // how bright the LED is
int fadeAmount = 5; // how many points to fade the LED by
// the setup routine runs once when you press reset:*

*void setup() {
 // declare pin 9 to be an output:
 pinMode(led, OUTPUT);
}*

// the loop routine runs over and over again forever:

*void loop() {
 // set the brightness of pin 9:
 analogWrite(led, brightness);
 // change the brightness for next time through the loop:
 brightness = brightness + fadeAmount;
 // reverse the direction of the fading at the ends of the fade:
 if (brightness == 0 || brightness == 255) {
 fadeAmount = -fadeAmount ;
 }
 delay(300); // wait for 30 milliseconds to see the dimming effect
}*

Arduino - Reading Analog Voltage (AnalogRead)

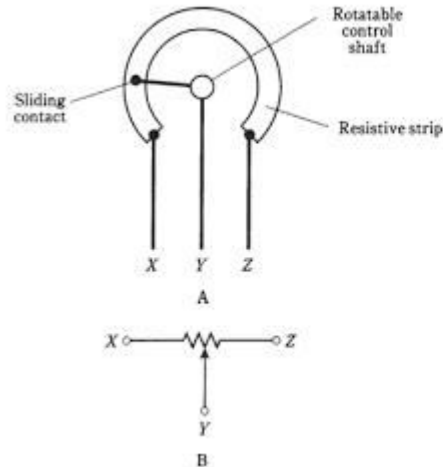
Analog input on analog pin 0

Reading analog voltage with `analogRead()` is super useful — especially when using sensors like potentiometers, light sensors (LDRs), or temperature sensors.

Wiring Example: Potentiometer to A0

If you're using a **potentiometer**, here's a basic wiring guide:

- **Center pin** of the potentiometer → **A0**
- One side pin → **5V**
- Other side pin → **GND**



Read and Display Analog Voltage

```
int analogPin = A0; // analog input pin
int analogValue = 0; // variable to store the
value
float voltage = 0.0; // variable to store the
voltage

void setup() {
  Serial.begin(9600); // Start the serial monitor
}

void loop() {
  analogValue = analogRead(analogPin);
  // Read the analog input (0-1023)
  voltage = analogValue * (5.0 / 1023.0); //
  Convert to voltage (0-5V)

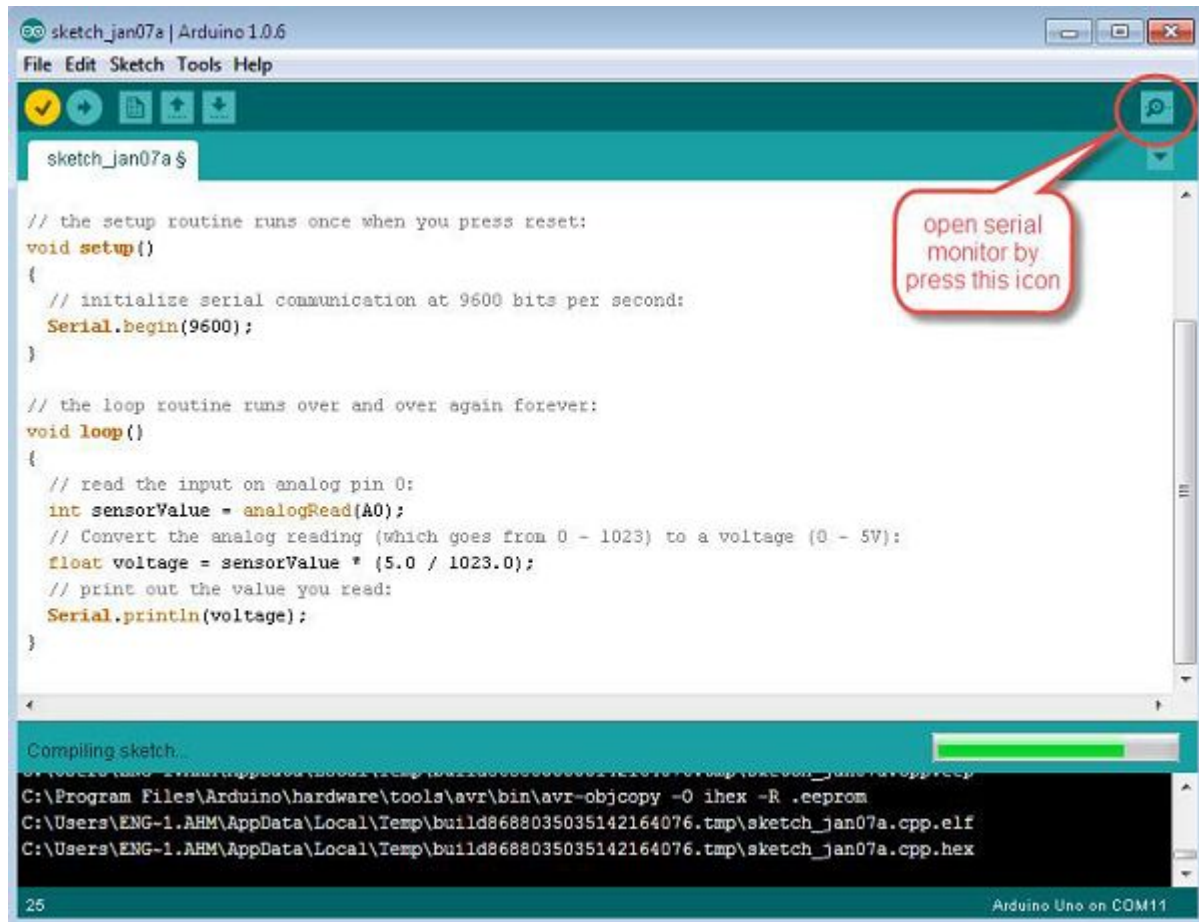
  Serial.print("Analog value: ");
  Serial.print(analogValue);
  Serial.print(" -> Voltage: ");
  Serial.print(voltage);
  Serial.println(" V");
  delay(500);
}
```

`analogRead()` reads a 10-bit value:
0 to 1023

Arduino Uno has a **5V ADC reference**
(default), so:

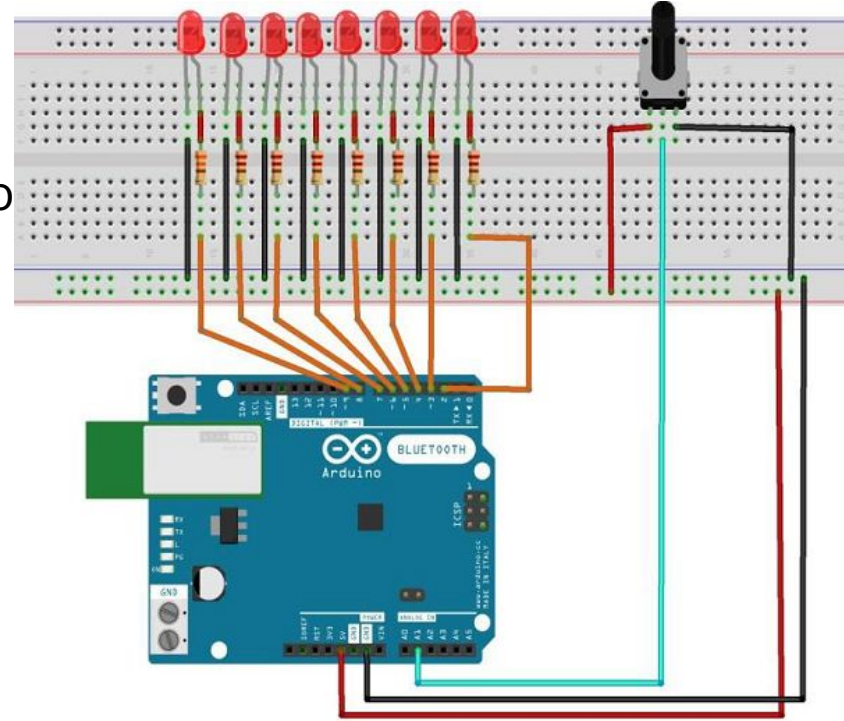
- 0 = 0V
- 1023 = 5V

The formula `voltage = analogValue * (5.0 / 1023.0)` converts that range to volts



Arduino - LED Bar Graph

To read an analog input at analog pin 0, convert the values from `analogRead()` into voltage, and print it out to the serial monitor of the Arduino Software (IDE).



The `map()` function in Arduino is **super handy** when you want to convert a value from one range into another.

map(value, fromLow, fromHigh, toLow, toHigh)

Read from a potentiometer:

int sensorValue = analogRead(A0);

To convert that value to a number of LEDs (0 to 8):

int ledLevel = map(sensorValue, 0, 1023, 0, 8);

It's like saying: "As this input goes from 0 to 1023, I want the output to go from 0 to 8."

Using `map(sensorValue, 0, 1023, 0, 8)`

Analog Reading (Range)	ledLevel	LEDs ON (Index in <code>ledPins[]</code>)
0 – 127	0	None
128 – 255	1	2
256 – 383	2	2, 3
384 – 511	3	2, 3, 4
512 – 639	4	2, 3, 4, 5
640 – 767	5	2, 3, 4, 5, 6
768 – 895	6	2, 3, 4, 5, 6, 7
896 – 1022	7	2, 3, 4, 5, 6, 7, 8
1023	8	2, 3, 4, 5, 6, 7, 8, 9

Arduino Code - LED Bar Graph

```
const int analogPin = A0; // the pin that the potentiometer is attached to  
const int ledCount = 8; // the number of LEDs in the bar graph  
int ledPins[] = {2, 3, 4, 5, 6, 7, 8, 9}; // an array of pin numbers to which LEDs are attached
```

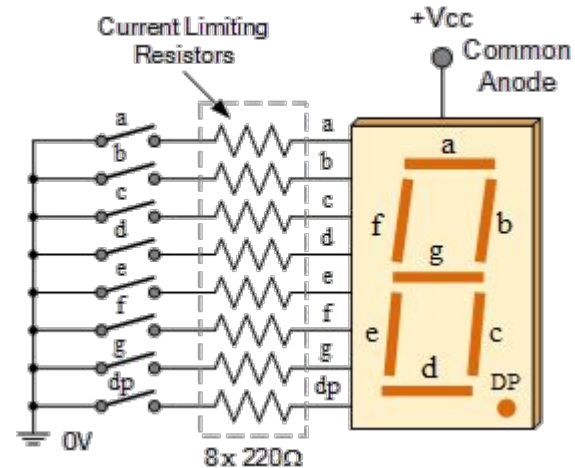
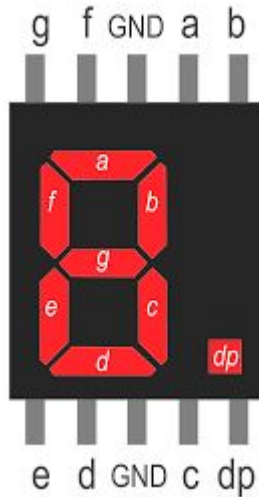
```
void setup() {  
  // loop over the pin array and set them all to output:  
  for (int thisLed = 0; thisLed < ledCount; thisLed++) {  
    pinMode(ledPins[thisLed], OUTPUT);  
  }  
}
```

```
void loop() {  
  // read the potentiometer:  
  int sensorReading = analogRead(analogPin);  
  // map the result to a range from 0 to the number of LEDs:  
  int ledLevel = map(sensorReading, 0, 1023, 0, ledCount);  
  // loop over the LED array:  
  for (int thisLed = 0; thisLed < ledCount; thisLed++) {  
    // if the array element's index is less than ledLevel,  
    // turn the pin for this element on:  
    if (thisLed < ledLevel) {  
      digitalWrite(ledPins[thisLed], HIGH);  
    }else { // turn off all pins higher than the ledLevel:  
      digitalWrite(ledPins[thisLed], LOW);  
    }  
  }  
}
```

7 segment led display using arduino

7-segment displays are classic for showing numbers (and even a few letters) on an Arduino.

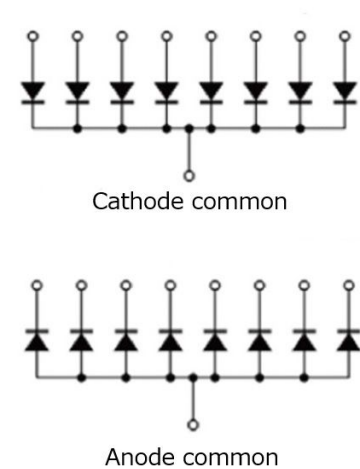
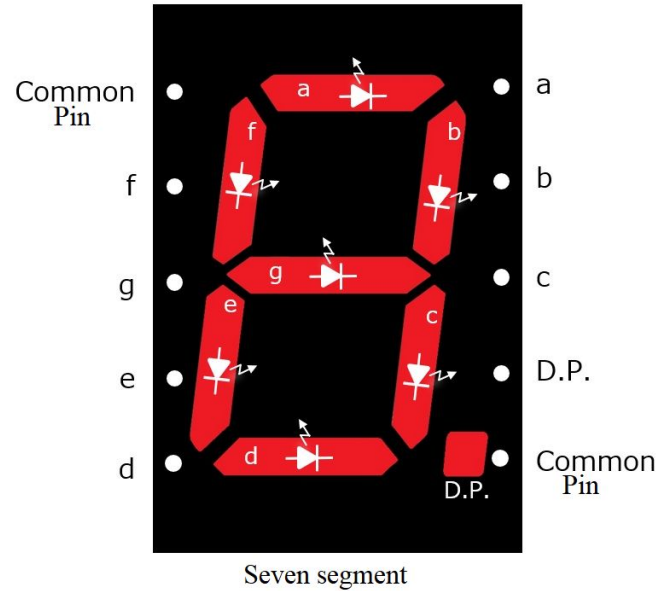
Each segment is a letter from **A to G**, sometimes **DP** for the decimal point.



The **common pin** connects to:

- **Cathode (GND)** for **common cathode**
- **Anode (+5V)** for **common anode**

Type	Segments Turn ON When...	Segment Pins Connect To
Common Cathode	Segment pin is given HIGH (5V)	Arduino digital pins
Common Anode	Segment pin is given LOW (GND)	Arduino digital pins



// A B C D E F G

{1, 1, 1, 1, 1, 1, 0}, // 0

{0, 1, 1, 0, 0, 0, 0}, // 1

{1, 1, 0, 1, 1, 0, 1}, // 2

{1, 1, 1, 1, 0, 0, 1}, // 3

{0, 1, 1, 0, 0, 1, 1}, // 4

{1, 0, 1, 1, 0, 1, 1}, // 5

{1, 0, 1, 1, 1, 1, 1}, // 6

{1, 1, 1, 0, 0, 0, 0}, // 7

{1, 1, 1, 1, 1, 1, 1}, // 8

{1, 1, 1, 1, 0, 1, 1} // 9

0 1 2 3 4
5 6 7 8 9

shutterstock.com - 2407695189

Example : Display the number 3

```
int segmentPins[] = {2, 3, 4, 5, 6, 7, 8}; // A to G

// segment pattern for digit '3' (A, B, C, D, G ON)
bool digits[10][7] = {
  // A B C D E F G
  {1, 1, 1, 1, 1, 1, 0}, // 0
  {0, 1, 1, 0, 0, 0, 0}, // 1
  {1, 1, 0, 1, 1, 0, 1}, // 2
  {1, 1, 1, 1, 0, 0, 1}, // 3
  {0, 1, 1, 0, 0, 1, 1}, // 4
  {1, 0, 1, 1, 0, 1, 1}, // 5
  {1, 0, 1, 1, 1, 1, 1}, // 6
  {1, 1, 1, 0, 0, 0, 0}, // 7
  {1, 1, 1, 1, 1, 1, 1}, // 8
  {1, 1, 1, 1, 0, 1, 1} // 9
};
```

```
void setup() {
  for (int i = 0; i < 7; i++) {
    pinMode ( segmentPins[i], OUTPUT);
  }
}

void loop() {
  int number = 3; // change this from 0 to 9

  for (int i = 0; i < 7; i++) {
    digitalWrite(segmentPins[i],
digits[number][i]);
  }

  delay(1000);
}
```

Example : Display the number 0-9

```
int segmentPins[] = {2, 3, 4, 5, 6, 7, 8}; // A, B,  
C, D, E, F, G
```

```
// Segment patterns for digits 0 to 9 (for  
common cathode)
```

```
//      A B C D E F G
```

```
bool digits[10][7] = {  
    {1, 1, 1, 1, 1, 1, 0}, // 0  
    {0, 1, 1, 0, 0, 0, 0}, // 1  
    {1, 1, 0, 1, 1, 0, 1}, // 2  
    {1, 1, 1, 1, 0, 0, 1}, // 3  
    {0, 1, 1, 0, 0, 1, 1}, // 4  
    {1, 0, 1, 1, 0, 1, 1}, // 5  
    {1, 0, 1, 1, 1, 1, 1}, // 6  
    {1, 1, 1, 0, 0, 0, 0}, // 7  
    {1, 1, 1, 1, 1, 1, 1}, // 8  
    {1, 1, 1, 1, 0, 1, 1} // 9  
};
```

```
void setup() {  
    for (int i = 0; i < 7; i++) {  
        pinMode(segmentPins[i], OUTPUT);  
    }  
}
```

```
void loop() {  
    for (int num = 0; num <= 9; num++) {  
        displayDigit(num);  
        delay(1000); // wait 1 second  
    }  
}
```

```
// Function to display a digit  
void displayDigit(int number) {  
    for (int i = 0; i < 7; i++) {  
        digitalWrite(segmentPins[i], digits[number][i]);  
    }  
}
```